

Параметры Запуска

Размер сетки: $N = 1024$

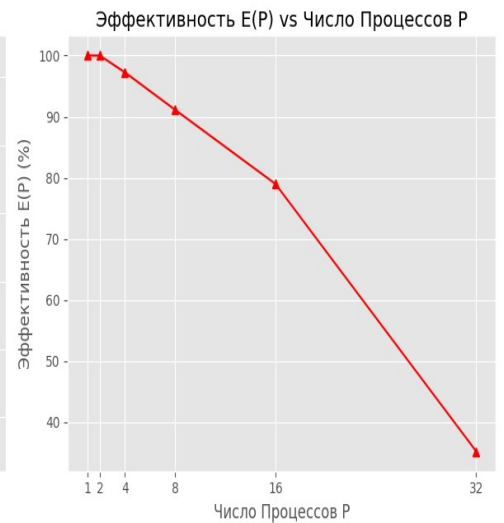
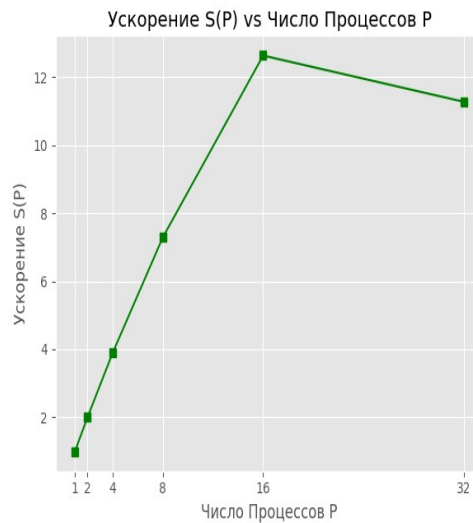
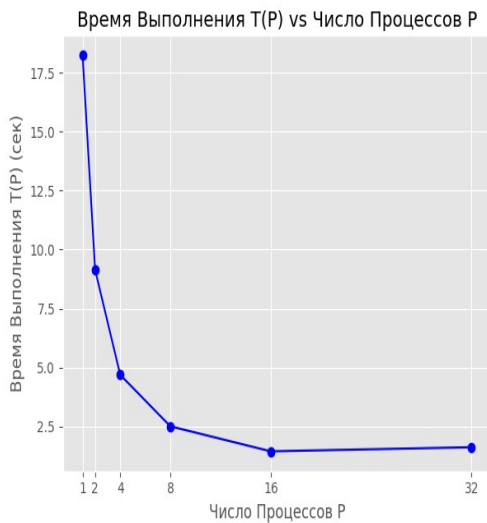
Число итераций: $MAX_ITER = 1000$

Число процессов: $P = \{1, 2, 4, 8, 16, 32\}$

Результаты

Ниже представлены результаты выполнения программы с различным числом процессов:

P	T(P) (сек)	S(P)	E(P) (%)
1	18.228846	1.0	100
2	9.133372	2.0	100
4	4.683925	3.89	97.25
8	2.501231	7.29	91.13
16	1.441353	12.64	79.0
32	1.614784	11.28	35.25



Выводы

- По мере увеличения числа процессов от 1 до 16 время выполнения значительно уменьшается, а ускорение приближается к линейной масштабируемости. Это указывает на эффективную параллелизацию до этого числа процессов.
- Эффективность снижается по мере увеличения числа процессов. Это связано с накладными расходами при межпроцессной коммуникацией.
- При 32 процессах время выполнения увеличивается по сравнению с 16 процессами, а эффективность резко падает до 35.25%. Это свидетельствует о том, что накладные расходы на коммуникацию превышают выгоды от параллельных вычислений при таком масштабе. Процессы тратят больше времени на обмен данными, чем на вычисления.

Выводы MPI-P

Отчеты **mpiP** показывают, что основное время MPI-операций связано с функцией `MPI_Allreduce`, которая занимает значительную часть времени выполнения (около 25–30%) и практически всю часть времени, затраченного на MPI-операции (~90–98%). Кроме того, значительное время тратится на обмен большими сообщениями через функции `MPI_Send` и `MPI_Recv`. Некоторые процессы демонстрируют значительно меньший процент времени, затраченного на MPI-операции (~24%), что может свидетельствовать о неравномерном распределении нагрузки или низкой активности на этих процессах.